Technical Supplement for "Dual-Expert Keys for Reducing False-Positive Retrieval in Lifelong Vision–Language Model Editing"

Anonymous submission

Scope

This document lists implementation details that were omitted or compressed in the main paper. All scientific claims, method equations for Dual-Expert Keys and adversarial visual binding, evaluation protocol sizes, and primary result tables remain in the main text. Reviewers are not required to consult this supplement.

Editor Configuration

Primary experiments use the BLIP-2 OPT-2.7B LiveEdit configuration in `configs/liveedit/blip2-opt-2.7b.yaml`. Relevant fields not tabulated in the main paper:

Field	Value
Retrieval module dimension	1024
Cross-attention heads	8
Editing query tokens (`eqe_n`)	4
LoRA rank / scale	4 / 5
LLM mid dimension	2560
Edit layer index	21 (OPT decoder)
Base learning rate	1×10^{-4}
LR cut iteration / rate	10000 / 0.1
Rel / Gen / Loc / soft / hard routing weights	all 1

Table 1: BLIP-2 LiveEdit YAML settings used for reported runs.

When patch grounding is enabled, vision logits mix Q-Former and DINOv2 patch scores with fuse weight α=0.5 (`patch_routing_alpha`). The DINOv2 encoder is frozen; only the PatchGroundingModule and MoE generators are trained for the DEK/AVBT path.

Probe-Builder Exact Rules

The conceptual image-swap and hard-negative probes are defined in the main paper. Here we record the exact pairing rules used by `tools/build_grounding_eval_json.py`.

Prompt skeleton. For a source question string s, normalize whitespace, lowercase, and take the first seven tokens. This skeleton indexes candidates for hard-negative preference.

Entity exclusion.

- VLKEB (`entity_mode=folder`): the entity id is the first path component of the image relative path.
- EVQA / EasyEdit (`entity_mode=coco`): the entity id is the numeric COCO image id parsed from the filename.

Partners must have a different entity id than the source edit.

Ranking. Among other-entity rows, assign skeleton score 2 for an exact skeleton match, 1 if the candidate appears in the same skeleton bucket list, else 0. Hard-negative candidates require score >0 when such partners exist; otherwise any other-entity row is allowed. Swap candidates are all other-entity rows, sorted by the same score then by index distance. The first candidate whose image file exists on disk is kept; hard-negative and swap partners are preferably distinct.

Drop rule. A row is dropped if no valid hard-negative or no valid swap partner image exists.

Packaged VLKEB eval overlay stats. For the shipped `data/VLKEB/grounding/grounding_eval_short.json` metadata: pairing seed 42; 3174 input rows retained as 3174 outputs; skeleton-match rates 0.696 (hard-negative) and 0.636 (swap); zero missing partners after ranking.

FPR Logging Procedure

The main paper defines false-positive retrieval (FPR) as the fraction of adversarial queries that activate at least one foreign expert. Operationally, with a built lifelong pool:

1. Apply sequential edits so each sample contributes one or more pool entries; maintain a map from pool index to sample id.
2. For each image-swap or hard-negative probe, call routing diagnostics (`probe_routing`) and record retrieved pool indices.
3. Mark the probe as a false-positive if any retrieved index maps to a sample id other than the query's own sample.
4. Aggregate the indicator over the evaluated adversarial probes (optionally separately for swap vs. hard-negative).

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

In the code supplement this is exposed via `test_grounding_eval.py -log_routing` and `tools/log_grounding_diagnostics.py`. Answer-level swap/hard-negative token accuracy is computed separately by the standard evaluation harness and is not used as the FPR definition.

AVBT Auxiliary Terms

Let $\mathcal{L}_{\text{pos}}$, $\mathcal{L}_{\text{neg}}$, λ, β, and hinge τ be as in the main paper Method section. On an adversarial image, write π^{adv} for the softmax over pool experts plus the prototype slot, and let

$$\pi_{\max}^{\text{wrong}} = \max_{i \notin \mathcal{O} \cup \{\text{prot}\}} \pi_i^{\text{adv}}$$

be the largest probability on a non-own expert. The informal $\mathcal{L}_{\text{aux}}$ used in primary runs (with `-avbl_contrastive`) is

$$\mathcal{L}_{\text{contrast}} = \text{ReLU}(\pi_{\max}^{\text{wrong}} - \gamma)\,, \qquad \gamma{=}0.1, \tag{1}$$

$$\mathcal{L}_{\text{targeted}} = -\log(1 - \pi_{\max}^{\text{wrong}})\,, \tag{2}$$

and $\mathcal{L}_{\text{aux}} = \mathcal{L}_{\text{contrast}} + \mathcal{L}_{\text{targeted}}$. Neighbor-mining and pool-buffer extras are disabled in the primary Dual-Expert Keys row (they appear only as negative ablations in the main paper).

Training Schedule Details

Reported Dual-Expert Keys checkpoints are obtained by fine-tuning from a lifelong LiveEdit baseline trained on original VLKEB edits (no DEK, no AVBT). Typical command-line settings for the DEK+AVBT stage:

- batch size 4; one training epoch over the probe-augmented VLKEB train JSON;
- random seed 42 for the headline run;
- `-lkpt` set to the baseline checkpoint path;
- `-patch_grounding -adversarial_grounding -avbl_contrastive -dek_dual_gate`;
- binding hyperparameters as in the main paper (λ=0.3, β=2.5, τ=0.3, text margin m=0).

Model weights, VLKEB/EVQA images, and trained checkpoints are not redistributed with the optional code archive; they must be obtained separately.

Code and Data Artifacts

Relative paths in the anonymous code and data package:

- `configs/liveedit/blip2-opt-2.7b.yaml` — editor hyperparameters above;
- `tools/build_grounding_eval_json.py` — probe construction;
- `data/VLKEB/grounding/` and `data/easy-edit-mm/grounding/` — probe-augmented JSON metadata (images excluded);
- `train_vllm_editor.py` — training entry point;
- `test_grounding_eval.py` — adversarial evaluation and optional routing logs;
- `editor/vllm_editors/liveedit/` — DEK / AVBT / patch-grounding implementation.